SimpleGrid problems compiled to d-DNNF with d4 (projected, no evidence)

Instance	Horizon	Circuit file	d4 time (s)	Total time (s)	CNF variables	CNF clauses	Projected variables	Circuit nodes	Circuit edges	Model count (verified)	Query with 2 obs. (s)	Tries
grid3x3-1	5	grid3x3-1.dnnf	0.04	–	141	818	98	43	81	18	0.001	1
grid3x3-2	5	grid3x3-2.dnnf	0.03	–	141	818	98	45	85	18	0.001	1
grid3x3-3	5	grid3x3-3.dnnf	0.04	–	141	818	98	45	85	18	0.001	1
grid3x3-4	5	grid3x3-4.dnnf	0.03	–	141	818	98	45	85	18	0.001	1
grid4x4-1	7	grid4x4-1.dnnf	0.04	–	400	2,609	291	228	451	140	0.003	1
grid4x4-2	7	grid4x4-2.dnnf	0.04	–	400	2,609	291	228	451	140	0.003	1
grid4x4-3	7	grid4x4-3.dnnf	0.04	–	400	2,609	291	228	451	140	0.003	1
grid4x4-4	7	grid4x4-4.dnnf	0.04	–	400	2,609	291	228	451	140	0.003	1
grid5x5-1	9	grid5x5-1.dnnf	0.14	–	865	5,994	644	2,606	5,208	2,100	0.007	1
grid5x5-2	8	grid5x5-2.dnnf	0.06	–	760	5,248	564	393	781	315	0.006	1
grid5x5-3	9	grid5x5-3.dnnf	0.10	–	865	5,994	644	1,032	2,059	1,372	0.007	1
grid5x5-4	8	grid5x5-4.dnnf	0.05	–	760	5,248	564	399	793	280	0.006	1
grid6x6-1	11	grid6x6-1.dnnf	1.52	–	1,596	11,477	1,205	24,754	49,539	36,510	0.018	1
grid6x6-2	11	grid6x6-2.dnnf	2.00	–	1,596	11,477	1,205	29,556	59,122	28,302	0.019	1
grid6x6-3	10	grid6x6-3.dnnf	0.29	–	1,440	10,333	1,085	3,860	7,717	3,570	0.013	1
grid6x6-4	11	grid6x6-4.dnnf	1.64	–	1,596	11,477	1,205	26,389	52,808	28,302	0.019	1
grid7x7-1	12	grid7x7-1.dnnf	6.78	–	2,436	17,936	1,854	73,184	146,525	96,558	0.035	1
grid7x7-2	12	grid7x7-2.dnnf	4.49	–	2,436	17,936	1,854	57,227	114,500	65,472	0.033	1
grid7x7-3	12	grid7x7-3.dnnf	6.62	–	2,436	17,936	1,854	91,315	182,661	86,856	0.035	1
grid7x7-4	12	grid7x7-4.dnnf	9.21	–	2,436	17,936	1,854	109,265	218,648	103,620	0.039	1
grid8x8-1	14	grid8x8-1.dnnf	43.67	59.5	3,808	28,561	2,919	340,768	682,049	1,170,962	0.123	1
grid8x8-2	15	grid8x8-2.dnnf	148.10	205.4	4,096	30,753	3,143	1,044,687	2,093,264	7,856,368	0.349	1
grid8x8-3	15	grid8x8-3.dnnf	103.20	140.2	4,096	30,753	3,143	794,002	1,589,508	4,160,312	0.198	1
grid8x8-4	15	grid8x8-4.dnnf	121.58	164.8	4,096	30,753	3,143	963,185	1,929,510	8,536,710	0.307	1
grid9x9-1	16	grid9x9-1.dnnf	491.81	543.6	5,616	42,712	4,328	2,834,282	5,677,195	16,913,390	0.503	2
grid9x9-2	16	grid9x9-2.dnnf	555.55	616.1	5,616	42,712	4,328	3,426,037	6,862,665	17,737,109	0.624	2
grid9x9-3	16	grid9x9-3.dnnf	305.29	401.3	5,616	42,712	4,328	1,862,208	3,728,865	8,095,152	0.333	1
grid9x9-4	16	grid9x9-4.dnnf	318.10	437.8	5,616	42,712	4,328	1,980,196	3,965,175	10,575,175	0.342	1
grid10x10-1	18	none	timeout (20 min)	–	7,920	60,893	6,129	unfinished; d4 at 2.3 GB	–	–	–	1
grid10x10-2	18	none	timeout (20 min)	–	7,920	60,893	6,129	unfinished; d4 at 2.2 GB	–	–	–	1
grid10x10-3	17	none	timeout (20 min)	–	7,460	57,317	5,769	unfinished; d4 at 4.6 GB	–	–	–	1
grid10x10-4	18	none	timeout (20 min)	–	7,920	60,893	6,129	unfinished; d4 at 4.5 GB	–	–	–	1

Each problem's evidence-free recognition theory — the problem at its recognition horizon, with the goal literals as the hypotheses, as recognize.sh builds it — compiled by marginals.sh `--solver` d4 `--project`. The projection keeps the drive actions and the final-slice goal atoms, and FiFO first verifies that they determine every other atom, so the projected circuit gives exact counts; any pinned observations are unit literals over the kept actions, so one saved circuit answers every evidence file by conditioning, without recompiling. CNF variables and clauses are those of the ground theory d4 is given; projected variables are the ones it keeps. d4 time is the d4 run alone; total time is the whole marginals.sh run (reading the theory, the projection check, d4, importing and saving the circuit, and the marginals of the unconditioned circuit). Circuit nodes and edges are those of d4's decision-DNNF, before FiFO smooths it. Runs: 1 compilation at a time, each limited to 30 minutes (3×3–7×7) or 20 minutes (8×8–10×10). SBCL ran with an 8 GB heap (3×3–7×7) or a 12 GB heap (8×8–10×10) and a 1 GB control stack, d4 with a 64 MB stack (the defaults, 1 GB, 2 MB and 8 MB, are too small for the larger circuits). The 8×8–10×10 runs were timed on the awake clock (CLOCK_UPTIME_RAW), which stops while the computer sleeps, so a suspend neither lengthens a time nor times a run out; their records also hold wall-clock time and CPU time (grid9x9-1, grid9x9-2, grid10x10-4: the computer slept during the run). The circuits were imported by FiFO's d4 importer as rewritten to use bit-vector scopes and iterative traversal (FiFO fd0d0b9); the earlier importer ran out of memory on circuits of about a million nodes. grid9x9-1, grid9x9-2 were imported by its successor, which builds the circuit in one pass into packed typed arrays; with fd0d0b9, SBCL's garbage collector crashed while importing them, after d4 had finished. Model count (verified) is the number of models of the compiled circuit (all literals set to 1 in the smoothed circuit), that is, the number of truck walks the theory allows. Every circuit kept was checked against an independent count of those walks by dynamic programming over the grid (H-1 steps, each a drive to a neighbouring cell or a wait, from p1-1 to a goal cell), which agrees with sharpSAT on every problem up to 7×7 (sharpSAT did not finish an 8×8 problem in 2.5 hours); the two counts were equal for every circuit in the table. Until d4v2 commit c11210d (2026-10-16), d4 had a heap overflow (ListLit::setListLit) that from 8×8 on made it crash, or silently return a wrong circuit, at random. The 8×8–10×10 rows are from the fixed d4; the 3×3–7×7 circuits were compiled before the fix, which did not affect them (every one passed the count check). The 3×3–7×7 rows give the run whose circuit passed the model-count check (grid3x3-1: correct on 2 of 2; grid3x3-2: correct on 2 of 2; grid3x3-3: correct on 2 of 2; grid3x3-4: correct on 2 of 2; grid4x4-1: correct on 2 of 2; grid4x4-2: correct on 2 of 2; grid4x4-3: correct on 2 of 2; grid4x4-4: correct on 2 of 2; grid5x5-1: correct on 2 of 2; grid5x5-2: correct on 2 of 2; grid5x5-3: correct on 2 of 2; grid5x5-4: correct on 2 of 2; grid6x6-1: correct on 2 of 2; grid6x6-2: correct on 2 of 2; grid6x6-3: correct on 2 of 2; grid6x6-4: correct on 2 of 2; grid7x7-1: correct on 2 of 2; grid7x7-2: correct on 2 of 2; grid7x7-3: correct on 2 of 2; grid7x7-4: correct on 2 of 2 runs); its d4 time is that run's alone, not counting failed runs. Their total time is not shown: it was measured with FiFO's earlier, quadratic evaluator (and partly two compilations at a time); and no queries were timed on them. Tries counts the runs of d4 on a problem, failed ones included — a run whose saved circuit a later check found wrong counts as failed. Query with 2 obs. is the time to recompute every marginal of the saved circuit conditioned on two observations — the first

two drives of a shortest plan to the problem's alphabetically first destination, pinned to steps 1 and 2 (evidence/PROBLEM-DESTINATION-obs2.txt) — as marginals.sh `--circuit --evidence-file` does: the observations are unit literals over kept atoms, so the circuit is clamped and re-evaluated, not recompiled. It is timed inside SBCL, the median of 5 repetitions, and excludes starting SBCL and loading FiFO (about 1.2 s) and reading the circuit file (0.001–8.80 s). Each answer was checked: both observed drives have probability 1, and the final-slice goal marginals sum to 1. FiFO evaluates the circuit in two linear passes; its backward pass through an AND node of k children uses prefix and suffix products (an earlier version recomputed each child's k-1 sibling products, k^2 multiplications, and took up to 7 s on 7×7 — smoothed AND nodes here have up to 1,007 children). FiFO's circuit-size cap (*ddnnf-node-limit*, 2,000,000 nodes by default) was raised to 50,000,000 for 8×8–10×10, since a smoothed import of a large d4 circuit exceeds the default. d4 holds its circuit in memory and writes it only when it finishes, so an unfinished circuit cannot be counted; its size is given as d4's peak memory (sampled every 2 s) before it was stopped. No 10×10 problem compiled within the limit, so larger grids were not tried.